

Structured Programming and CRR Code Lab

FRE6883 – Recitation 2

Complete it, trace it, test it, explain it

Today we rebuild one pricer from small testable pieces

Model data S_0, U, D, R, N, K



Stock-price nodes



Terminal payoffs



Backward induction



Option price $H(0, 0)$

Your job on every activity

Predict the missing operation, complete the code, trace one example, and name one test that could expose a bug.

The goal is not to memorize the final function. It is to make every line explainable.

Three control structures are enough to express the algorithm

Sequence

```
read inputs;  
compute node;  
print result;
```

One statement follows another.

Condition

```
if (invalid)  
{  
    return -1;  
}
```

The data decides the path.

Iteration

```
for (int i = 0;  
     i <= n; ++i)  
{  
    visit(i);  
}
```

The loop repeats one rule.

Warm-up: point to one of each structure in the CRR procedure.

01 | Compute one stock-price node – Problem

GUIDED CODE Formula to function

Complete together

At step n and node i ,

$$S(n, i) = S_0 U^i D^{n-i}, \quad 0 \leq i \leq n.$$

```
double CalculateAssetPrice(  
    double S0, double U, double D, int n, int i)  
{  
    return /* complete the expression */;  
}
```

Hand check

$S_0 = 106$, $U = 1.15125$, $D = 0.86862$, $n = 3$, $i = 2$ should give approximately 122.03.

Think first: Which exponent counts up moves? Which counts down moves?

01 | Compute one stock-price node – Solution

```
double CalculateAssetPrice(  
    double S0, double U, double D, int n, int i)  
{  
    return S0 * std::pow(U, i)  
        * std::pow(D, n - i);  
}
```

The idea

i counts up moves; $n - i$ counts down moves. Their sum is always n .

Boundary nodes

$i = 0$ gives $S_0 D^n$; $i = n$ gives $S_0 U^n$.

Required header

`std::pow` is declared in `<cmath>`.

02 | Visit every node exactly once – Problem

GUIDED CODE Nested loops

Complete the bounds

For $N = 3$, the valid node pairs are

(0, 0); (1, 0), (1, 1); (2, 0), (2, 1), (2, 2); (3, 0), (3, 1), (3, 2), (3, 3).

```
for (int n = /* start */; n /* condition */ N; ++n)
{
    for (int i = /* start */; i /* condition */ n; ++i)
    {
        std::cout << CalculateAssetPrice(
            S0, U, D, n, i) << std::endl;
    }
}
```

Think first: How many nodes should the code print when $N = 3$?

02 | Visit every node exactly once – Solution

```
for (int n = 0; n <= N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        std::cout << CalculateAssetPrice(
            S0, U, D, n, i) << std::endl;
    }
}
```

Outer loop

Selects one time step from 0 through N .

Inner loop

Visits its $n + 1$ valid nodes, from 0 through n .

Node count

$1 + 2 + 3 + 4 = 10$ when $N = 3$; in general $(N + 1)(N + 2)/2$.

03 | Store the visited nodes in one array – Problem

GUIDED CODE Array + running index

Complete three blanks

```
const int MAX_N = 8;
const int SIZE = (MAX_N + 1) * (MAX_N + 2) / 2;
double prices[SIZE] = { 0.0 };

int index = /* ? */;
for (int n = 0; n <= MAX_N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        prices[index] = CalculateAssetPrice(
            S0, U, D, n, i);
        /* advance index */;
    }
}
```

Think first: What is the first valid array index? What is the last?

03 | Store the visited nodes in one array – Solution

```
const int MAX_N = 8;
const int SIZE = (MAX_N + 1)
                 * (MAX_N + 2) / 2;
double prices[SIZE] = { 0.0 };

int index = 0;
for (int n = 0; n <= MAX_N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        prices[index] = CalculateAssetPrice(
            S0, U, D, n, i);
        ++index;
    }
}
```

Capacity

Steps 0 through 8 contain
 $9 \cdot 10/2 = 45$ nodes.

Invariant

Before each write, `index` identifies the next free slot.

Safety

The final write uses index 44; then `index` becomes 45.

A loop generates nodes; an array remembers them

Computation

`CalculateAssetPrice` translates one mathematical rule into one reusable operation.

Traversal

The nested loops generate every valid pair (n, i) .

Storage

The array preserves results after the loop moves to the next node.

Check the design

Can we read `prices[SIZE]`? Why or why not?

04 | Reject invalid model data – Problem

GUIDED CODE Conditions + early return

Complete the no-arbitrage test

Valid data must satisfy $S_0 > 0$ and $0 < D < R < U$.

```
int ValidateInputData(  
    const double& S0, const double& U,  
    const double& D, const double& R)  
{  
    if (S0 <= 0.0 || D <= 0.0 || U <= D)  
    {  
        return -1;  
    }  
    if (/* reject R outside (D,U) */)   
    {  
        return -1;  
    }  
    return 0;  
}
```

Test: should $D = 0.9$, $R = 0.9$, $U = 1.1$ pass?

04 | Reject invalid model data – Solution

```
int ValidateInputData(  
    const double& S0, const double& U,  
    const double& D, const double& R)  
{  
    if (S0 <= 0.0 || D <= 0.0 || U <= D)  
    {  
        return -1;  
    }  
    if (R <= D || R >= U)  
    {  
        return -1;  
    }  
    return 0;  
}
```

Boundary matters

Equality creates an invalid boundary;
use $\leq$ and $\geq$.

Useful test

With $D = R = 0.9$, the function
returns -1.

Why early return

Invalid inputs cannot reach the
pricing calculation.

Functions give each financial rule one clear home

BinomialTreeModel.h

```
#pragma once
namespace fre {
double RiskNeutProb(
    double U, double D, double R);
double CalculateAssetPrice(
    double S0, double U, double D,
    int n, int i);
}
```

Declarations tell other files what exists.

BinomialTreeModel.cpp

```
#include "BinomialTreeModel.h"
namespace fre {
// definitions belong here
}
```

Definitions contain the executable rules.

Design question: which file should include `<cmath>`?

05 | Implement the risk-neutral weight – Problem

GUIDED CODE Small pure function

Complete and hand-check

$$q = \frac{R - D}{U - D}.$$

```
namespace fre {  
  
double RiskNeutProb(double U, double D, double R)  
{  
    return /* ? */;  
}  
  
}
```

Hand check

For $U = 1.2$, $D = 0.8$, $R = 1.1$, the result should be 0.75.

Think first: Which validation condition guarantees that $0 < q < 1$?

05 | Implement the risk-neutral weight – Solution

```
namespace fre {  
  
double RiskNeutProb(double U, double D, double R)  
{  
    return (R - D) / (U - D);  
}  
  
}
```

Parentheses

They preserve the numerator and denominator from the formula.

Test value

$$q = (1.1 - 0.8) / (1.2 - 0.8) = 0.75.$$

Precondition

$D < R < U$ places $R - D$ strictly between 0 and $U - D$.

06 | Predict value and reference effects – Problem

GUIDED CODE Trace before running

Fill the final output

```
void Foo(int a, int& b)
{
    ++a;
    ++b;
    std::cout << a << " " << b << std::endl;
}

int main()
{
    int x = 1, y = 1;
    Foo(x, y);
    std::cout << x << " " << y << std::endl;
}
```

Complete the trace

Inside Foo: __ __ Back in main: __ __

06 | Predict value and reference effects – Solution

Observed output

Inside Foo: 2 2

Back in main: 1 2

```
int a
```

The function receives a copy of x. Incrementing the copy does not change x.

```
int& b
```

The function receives an alias for y. Incrementing b changes y.

```
const int& b
```

The alias could be read but not incremented; ++b would fail to compile.

07 | Return four inputs through references – Problem

GUIDED CODE Output parameters

Complete the reads and success code

```
int GetInputData(  
    double& S0, double& U, double& D, double& R)  
{  
    std::cout << "Enter S0 U D R: ";  
    std::cin >> /* four output parameters */;  
  
    if (ValidateInputData(S0, U, D, R) == -1)  
    {  
        return /* failure */;  
    }  
    return /* success */;  
}
```

Call site

```
double s0=0.0, u=0.0, d=0.0, r=0.0;  
int status = fre::GetInputData(s0, u, d, r);
```

07 | Return four inputs through references – Solution

```
int GetInputData(  
    double& S0, double& U, double& D, double& R)  
{  
    std::cout << "Enter S0 U D R: ";  
    std::cin >> S0 >> U >> D >> R;  
  
    if (ValidateInputData(S0, U, D, R) == -1)  
    {  
        return -1;  
    }  
    return 0;  
}
```

Why references

One function call updates four variables owned by the caller.

Meaningful status

0 means success; -1 means invalid model data.

Caller responsibility

Check the status before attempting any pricing.

The interfaces now describe the data flow

Read or compute

`GetInputData` changes caller variables.
`CalculateAssetPrice` returns one computed value.

`RiskNeutProb` returns one computed value.

Protect the calculation

`ValidateInputData` checks model invariants before a denominator, probability, or tree is constructed.

Say it aloud

Which functions need references, and which do not?

08 | Implement a European call payoff – Problem

GUIDED CODE Conditional rule

Complete the function

$$h(z) = (z - K)^+ = \max(z - K, 0).$$

```
double CallPayoff(double z, double K)
{
    if (/* in the money */)
    {
        return /* positive payoff */;
    }
    return /* otherwise */;
}
```

Acceptance cases for $K = 100$

$z = 80 \rightarrow 0$, $z = 100 \rightarrow 0$, $z = 120 \rightarrow 20$.

Boundary question: Should the condition use $>$ or $>=$?

08 | Implement a European call payoff – Solution

```
double CallPayoff(double z, double K)
{
    if (z > K)
    {
        return z - K;
    }
    return 0.0;
}
```

At the strike

$z - K = 0$, so both $>$ and $\geq$ return zero.
 $>$ states “positive payoff” directly.

One responsibility

The function knows the payoff rule, not how stock prices are generated.

Easy to test

No array or loop is required to verify the three cases.

09 | Build the terminal payoff row – Problem

GUIDED CODE Loop + two function calls

Complete the row

At expiry N , each array entry is $H(N, i) = h(S(N, i))$.

```
double Price[SIZE] = { 0.0 };

for (int i = /* first node */; i /* last node */ N; ++i)
{
    double stock = /* compute S(N,i) */;
    Price[i] = /* apply the payoff */;
}
```

Trace this case

$S_0 = 100$, $U = 1.2$, $D = 0.8$, $K = 100$, $N = 2$.

Complete the stock row [__, __, __] and payoff row [__, __, __].

09 | Build the terminal payoff row – Solution

```
double Price[SIZE] = { 0.0 };  
  
for (int i = 0; i <= N; ++i)  
{  
    double stock = CalculateAssetPrice(  
        S0, U, D, N, i);  
    Price[i] = CallPayoff(stock, K);  
}
```

Stock row

[64, 96, 144] for $i = 0, 1, 2$.

Payoff row

[0, 0, 44] for a call struck at 100.

Invariant

After the loop, indices 0 through N hold the complete expiry row.

Backward induction shrinks one row at a time

For $q = 0.75$ and $R = 1.1$, begin with the terminal row $[0, 0, 44]$.

Step	Active values	Update rule
$n = 2$	$[0, 0, 44]$	terminal call payoffs
$n = 1$	$[0, 30]$	$H(1, i) = \frac{0.75H(2, i+1) + 0.25H(2, i)}{1.1}$
$n = 0$	$[20.4545]$	same local rule

Why one array is enough

After computing row n , the old row $n + 1$ is never needed again. We overwrite it from left to right.

10 | Complete backward induction – Problem

GUIDED CODE Two loops + in-place update

Complete four blanks

```
for (int n = /* first backward step */;
    n /* continue through root */ 0; --n)
{
    for (int i = 0; i /* nodes on row n */ n; ++i)
    {
        Price[i] =
            (q * /* up child */
             + (1.0 - q) * /* down child */) / R;
    }
}
return Price[0];
```

Index map

At node (n, i) , the down child is $\text{Price}[i]$ and the up child is $\text{Price}[i+1]$.

10 | Complete backward induction – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
return Price[0];
```

Outer loop

Expiry is already known, so begin at $N - 1$ and finish at the root.

Inner loop

Row n contains indices 0 through n .

Final state

After $n = 0$, $\text{Price}[0]$ is $H(0, 0)$.

11 | Find the overwrite bug – Problem

GUIDED CODE Data dependency

The formulas are right; the result is wrong

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = n; i >= 0; --i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
```

Trace only row $n = 1$

Start from `Price = [0, 0, 44]`. Which entry is overwritten first? Which later calculation reads that changed entry as if it were still a child?

Repair: Change only the inner-loop header.

11 | Find the overwrite bug – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
```

What went wrong

With decreasing i , $\text{Price}[1]$ changes before node $(1,0)$ reads it as its up child.

Why increasing works

Updating $\text{Price}[0]$ cannot damage $\text{Price}[1]$, which is still needed next.

General lesson

When overwriting an array, loop direction is part of correctness.

12 | Assemble PriceByCRR – Problem A

GUIDED CODE Guard + initialization

Complete the preconditions

```
double PriceByCRR(double S0, double U, double D,
                  double R, const int N, double K)
{
    const int SIZE = 9;
    if (N < 1 || N >= SIZE || S0 <= 0.0 || K < 0.0
        || /* require 0 < D < R < U */)
    {
        std::cerr << "Invalid model data" << std::endl;
        return -1.0;
    }

    double q = /* helper call */;
    double Price[SIZE] = { 0.0 };
    // Next: construct the terminal row.
```

Design check

Why does `SIZE = 9` allow $N = 8$ but reject $N = 9$?

12 | Assemble PriceByCRR – Problem B

GUIDED CODE Terminal row + backward pass

Connect the completed pieces

```
for (int i = 0; i <= N; ++i)
{
    Price[i] = /* payoff at S(N,i) */;
}

for (int n = /* ? */; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] = /* one CRR update */;
    }
}
return /* root value */;
}
```

Before the reveal

Point to the base case, the recurrence, and the final answer in this function.

12 | Assemble PriceByCRR – Solution A

```
double PriceByCRR(double S0, double U, double D,
                  double R, const int N, double K)
{
    const int SIZE = 9;
    if (N < 1 || N >= SIZE || S0 <= 0.0 || K < 0.0
        || D <= 0.0 || U <= D || R <= D || R >= U)
    {
        std::cerr << "Invalid model data" << std::endl;
        return -1.0;
    }

    double q = RiskNeutProb(U, D, R);
    double Price[SIZE] = { 0.0 };
    for (int i = 0; i <= N; ++i)
    {
        Price[i] = CallPayoff(
            CalculateAssetPrice(S0, U, D, N, i), K);
    }
}
```

The validation establishes safe ranges; the terminal loop establishes the base case.

12 | Assemble PriceByCRR – Solution B

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
return Price[0];
}
```

Shared example

$N = 2$ returns approximately 20.4545 for
 $S_0 = 100$, $U = 1.2$, $D = 0.8$, $R = 1.1$,
 $K = 100$.

Explain the memory

At any moment, the active prefix of
Price represents one complete time level.

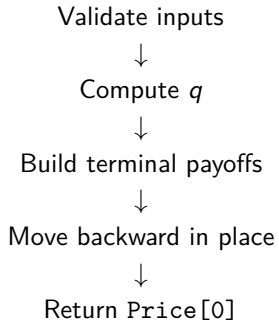
Tests should target boundaries and invariants

Case	Change	Expected observation	Bug targeted
Baseline	$S_0 = 100, U = 1.2, D = .8, R = 1.1, K = 100, N = 2$	20.4545	full data flow
At strike	call payoff $z = K = 100$	0	payoff boundary
No-arbitrage edge	$R = D$	rejected	wrong inequality
Capacity edge	$N = 9$ with SIZE=9	rejected	out-of-bounds write
Zero strike	set $K = 0$	equal to S_0	discount/update error

A useful explanation has three parts

State the input, predict the result or invariant, then identify the line that would be suspect if the test failed.

One algorithm now has a line-by-line explanation



Exit ticket

1. Why must R lie strictly between D and U ?
2. Why does the inner CRR loop move from left to right?
3. Which function would you test first after changing the payoff from call to put?

If you can answer all three, you can debug the pricer rather than merely rerun it.